

Shiv Nadar University Chennai
CS3807 – Deep Learning Laboratory
Experiment 3

Implementation of Convolutional Neural Networks (CNNs) for Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Hemanth Kumar R
24011101035
B.Tech AI&DS 'A'

1. Objective

To understand the working principle of Convolutional Neural Networks by implementing convolution, pooling, feature map visualization, and image classification using TensorFlow/Keras.

2. Learning Outcomes

Students will be able to

1. Explain the convolution operation.
2. Compute output dimensions using stride and padding.
3. Visualize feature maps.
4. Implement max pooling and average pooling.
5. Calculate CNN parameters.
6. Build a CNN for image classification.
7. Evaluate CNN performance.

3. Background Theory

A Convolutional Neural Network consists of

- Convolution Layer
- Activation Layer
- Pooling Layer
- Fully Connected Layer

The convolution operation is

$$Y(i, j) = \sum_m \sum_n X(i + m, j + n) K(m, n)$$

where

- X : Input Image
- K : Kernel (Filter)
- Y : Feature Map

The output feature map size is

$$\frac{N - F + 2P}{S} + 1$$

where N Input Size
 F Filter Size
 P Padding
 S Stride

3.1 Common Convolution Kernels

A kernel (or filter) is a small matrix that slides over an input image to extract important visual features such as edges, textures, corners, and smooth regions. During convolution, the kernel is multiplied element-wise with the corresponding image pixels, and the results are summed to produce a single value in the output feature map. Different kernels highlight different image characteristics.

1. Identity Kernel

The identity kernel preserves the original image without modifying its pixel values.

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Purpose:

- Preserves the original image.
- Used for understanding the convolution operation.

2. Sobel-X Kernel (Vertical Edge Detection)

The Sobel-X kernel detects **vertical edges** by measuring intensity changes along the horizontal direction.

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Applications:

- Object detection
- Lane detection
- Face recognition

3. Sobel-Y Kernel (Horizontal Edge Detection)

The Sobel-Y kernel detects **horizontal edges** by measuring intensity changes along the vertical direction.

$$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Applications:

- Text detection
- Boundary extraction
- Medical image analysis

4. Laplacian Kernel

The Laplacian kernel detects edges in all directions using the second-order derivative of image intensity.

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Applications:

- Edge enhancement

- Satellite image analysis
- Medical imaging

5. Sharpening Kernel

This kernel enhances fine details by emphasizing high-frequency components.

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Applications:

- Image enhancement
- Optical Character Recognition (OCR)
- Face recognition

6. Box Blur Kernel

The Box Blur kernel smooths an image by replacing each pixel with the average of its neighboring pixels.

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Applications:

- Noise removal
- Image smoothing
- Image preprocessing

7. Gaussian Blur Kernel

Gaussian blur smooths the image while preserving the overall structure better than a simple average filter.

$$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

Applications:

- Noise reduction
- Computer vision preprocessing
- Feature extraction

8. Emboss Kernel

The Emboss kernel creates a three-dimensional appearance by emphasizing directional changes in intensity.

$$\begin{bmatrix} -2 & -1 & 0 \\ -1 & 1 & 1 \\ 0 & 1 & 2 \end{bmatrix}$$

Applications:

- Texture analysis
- Artistic image effects

9. Outline Detection Kernel

This kernel highlights object boundaries by emphasizing regions with rapid intensity changes.

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

Applications:

- Image segmentation
- Boundary detection
- Shape extraction

10. Motion Blur Kernel

This kernel simulates motion blur along the diagonal direction.

$$\frac{1}{5} \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

Applications:

- Motion simulation
- Image restoration
- Video processing

Summary of Common Kernels

Kernel	Size	Purpose	Applications
Identity	3×3	Preserve original image	Baseline comparison
Sobel-X	3×3	Detect vertical edges	Object detection
Sobel-Y	3×3	Detect horizontal edges	Boundary detection
Laplacian	3×3	Detect edges in all directions	Medical imaging
Sharpen	3×3	Enhance details	Image enhancement
Box Blur	3×3	Smooth image	Noise removal
Gaussian Blur	3×3	Reduce noise	Computer vision
Emboss	3×3	3D effect	Artistic filtering
Outline	3×3	Boundary extraction	Segmentation
Motion Blur	5×5	Simulate motion	Video processing

4. Numerical Example 1: Convolution

Consider the following input image.

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

Kernel

$$K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Output

First position

$$1(1) + 2(0) + 4(0) + 5(1) = 6$$

Second position

$$2(1) + 3(0) + 5(0) + 6(1) = 8$$

Third position

$$4(1) + 5(0) + 7(0) + 8(1) = 12$$

Fourth position

$$5(1) + 6(0) + 8(0) + 9(1) = 14$$

Feature Map

$$\begin{bmatrix} 6 & 8 \\ 12 & 14 \end{bmatrix}$$

5. Numerical Example 2: Max Pooling

Feature map

$$\begin{bmatrix} 1 & 5 & 2 & 3 \\ 7 & 8 & 1 & 0 \\ 4 & 6 & 9 & 5 \\ 2 & 3 & 1 & 8 \end{bmatrix}$$

Using a 2×2 max pooling filter,

Window 1

$$\begin{bmatrix} 1 & 5 \\ 7 & 8 \end{bmatrix} \rightarrow 8$$

Window 2

$$\begin{bmatrix} 2 & 3 \\ 1 & 0 \end{bmatrix} \rightarrow 3$$

Window 3

$$\begin{bmatrix} 4 & 6 \\ 2 & 3 \end{bmatrix} \rightarrow 6$$

Window 4

$$\begin{bmatrix} 9 & 5 \\ 1 & 8 \end{bmatrix} \rightarrow 9$$

Output

$$\begin{bmatrix} 8 & 3 \\ 6 & 9 \end{bmatrix}$$

6. Numerical Example 3: Parameter Calculation

Suppose

Input Image

$$32 \times 32 \times 3$$

Convolution Layer

- Filters = 16
- Kernel = 3×3

Parameters

$$\begin{aligned}(3 \times 3 \times 3 + 1) \times 16 \\ = (27 + 1) \times 16 \\ = 448\end{aligned}$$

Therefore,
Total trainable parameters = 448.

7. Dataset

Dataset: CIFAR-10

- Training Images : 50,000
- Testing Images : 10,000
- Classes : 10
- Image Size : $32 \times 32 \times 3$

8. Experimental Procedure

Task 1

Load CIFAR-10.

- Display ten sample images.
- Print dataset dimensions.
- Plot class distribution.

Task 2

Implement a convolution layer.

Compare

- Kernel = 3×3
- Kernel = 5×5
- Kernel = 7×7

Record feature map sizes.

Task 3

Study hyperparameters.

Compare

- Stride = 1
- Stride = 2
- Padding = Same
- Padding = Valid

Compute output dimensions.

Task 4

Visualize feature maps after the first convolution layer.

Display at least eight feature maps.

Task 5

Compare

- Max Pooling
- Average Pooling

Compare output size and accuracy.

Task 6

Construct the following CNN.

Input $\rightarrow$ Conv $\rightarrow$ ReLU $\rightarrow$ MaxPool $\rightarrow$ Conv $\rightarrow$ ReLU $\rightarrow$ MaxPool $\rightarrow$ Flatten $\rightarrow$ Dense $\rightarrow$ Softmax

Train for

- Optimizer : Adam
- Epochs : 20
- Batch Size : 32

Task 7

Evaluate

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

9. Source Code

The complete implementation of the Convolutional Neural Network (CNN) for image classification developed for this experiment is available in the GitHub repository below. The repository contains the Jupyter Notebook, source code, dataset-related files, and generated results.

Repository Link

<https://github.com/AI-MANIAC007/Hands-On-Deep-Learning/>

10. Mandatory Plots

1. Sample Images

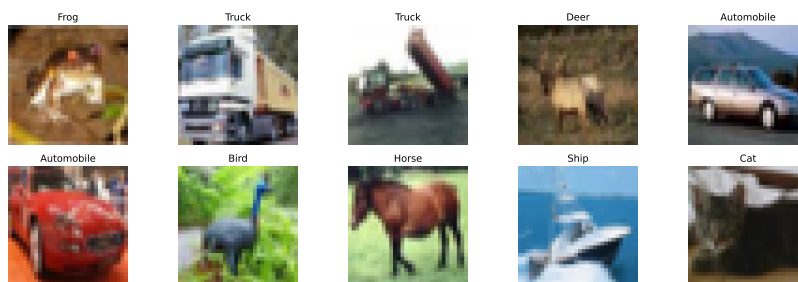


Figure 1: Sample images from the CIFAR-10 training dataset.

Based on the example pictures, it is clear that CIFAR-10 pictures consist of objects that belong to different classes, such as frog, truck, automobile, bird, horse, ship, and cat. Pictures are only 32×32 pixels, and thus the objects depicted are not always highly detailed.

2. Class Distribution

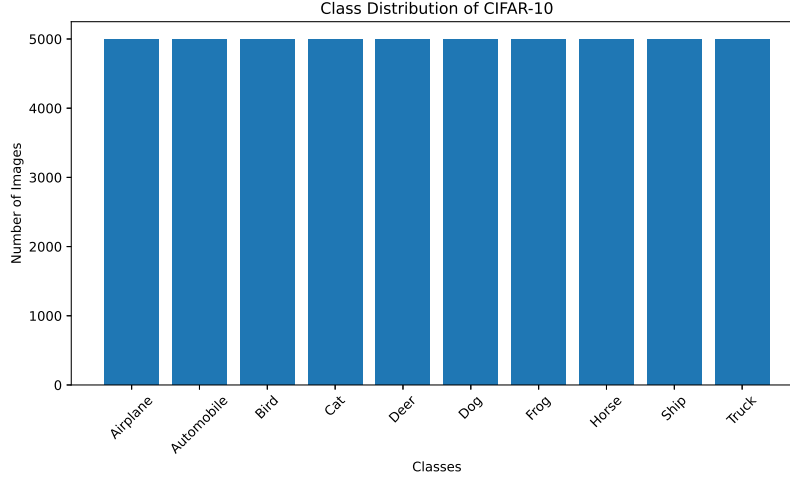


Figure 2: Class distribution of the CIFAR-10 training dataset.

As per the distribution chart for the class, it can be observed that there are almost equal numbers of training images for all ten classes, around 5000. Hence, the dataset is very much balanced, which implies that there will be no chance of any biasness for any particular class.

3. Training and Validation Accuracy

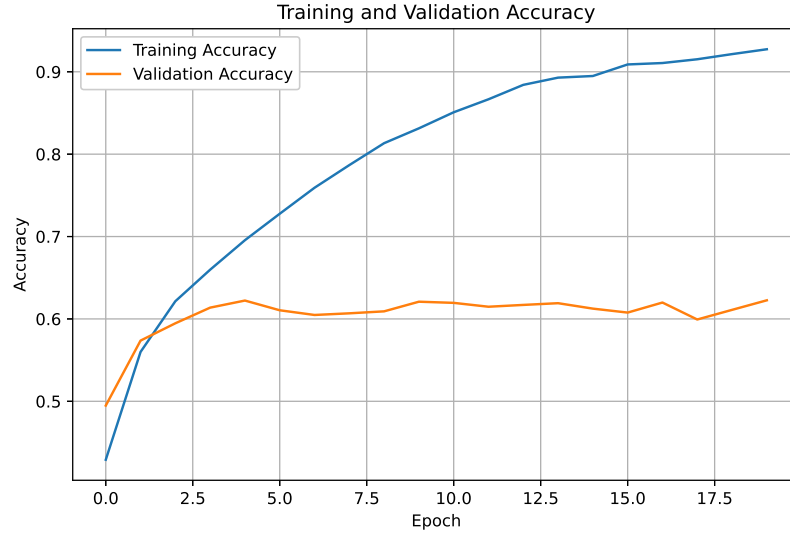


Figure 3: Training and validation accuracy over 20 epochs.

With an increase in the number of epochs, the training accuracy increases, which means that the CNN is learning patterns from the training data. But in the latter epochs, the validation accuracy does not come up to the level of training accuracy and is significantly lower. This shows that the model is not performing well on new data and is experiencing overfitting.

4. Training and Validation Loss

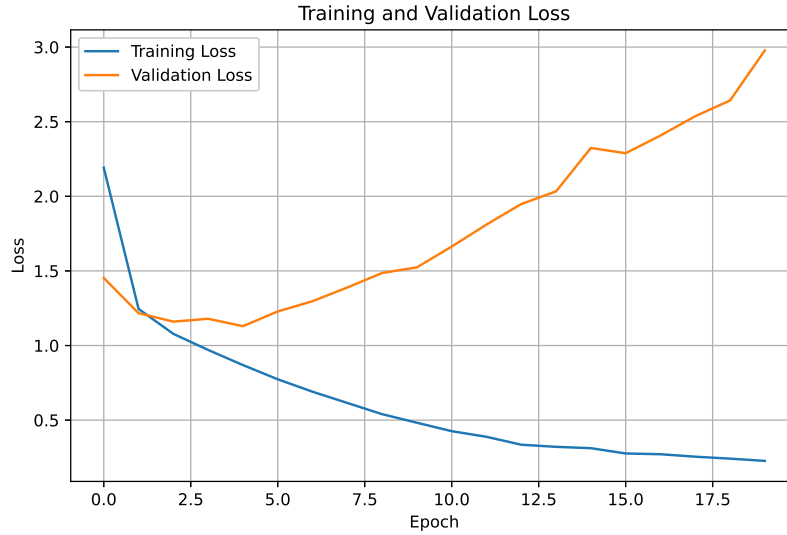


Figure 4: Training and validation loss over 20 epochs.

Training loss gets lower as the training continues, which suggests that the predictions of the model based on the training data get better. The validation loss is not following training loss in the late epochs, which shows that although the model gets better on training data, it does not improve when applied to other images.

5. Feature Maps

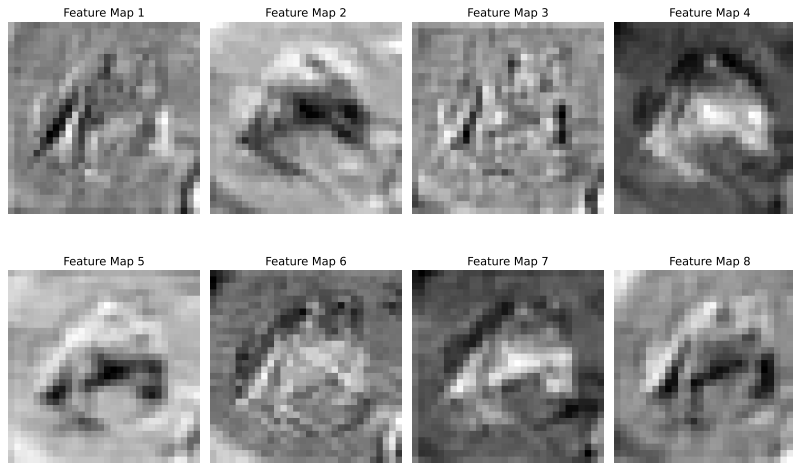


Figure 5: Feature maps obtained from the first convolution layer.

From the feature maps above, it is clear that the filters generate different activation maps from the same input image. There are some maps that capture the presence of strong edges or borders, whereas there are other maps that capture different local features of the image.

6. Confusion Matrix

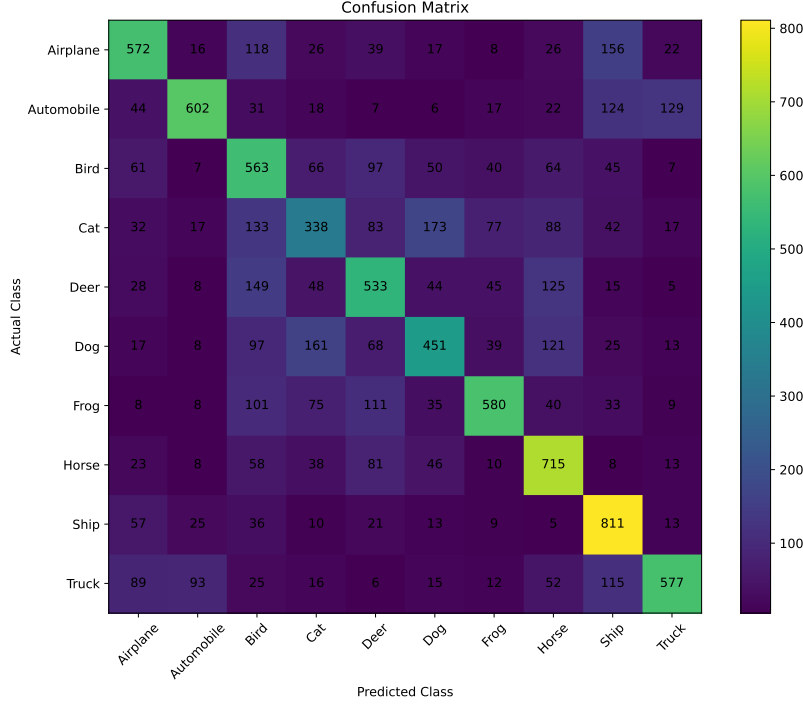


Figure 6: Confusion matrix for the CNN classification results.

As can be seen from the confusion matrix above, some classes are accurately classified as denoted by the values close to one on the diagonal line. Nonetheless, there are many instances of confusion between visually similar classes where Cat is misclassified as Dog 173 times, whereas Dog is misclassified as Cat 161 times. Likewise, there is a little bit of confusion between Airplane and Ship where 156 Airplanes are incorrectly classified as Ships.

11. Additional Exercises

1. Calculate the output size for a 64×64 image using a 5×5 kernel with stride 2 and padding 2.

The output size is calculated using

$$O = \left\lfloor \frac{N - F + 2P}{S} \right\rfloor + 1$$

where $N = 64$, $F = 5$, $P = 2$, and $S = 2$.

$$O = \left\lfloor \frac{64 - 5 + 2(2)}{2} \right\rfloor + 1$$

$$O = \left\lfloor \frac{63}{2} \right\rfloor + 1 = 31 + 1 = 32$$

Therefore, the output feature map size is

$$32 \times 32$$

The padding helps maintain a larger spatial dimension, while the stride of 2 reduces the output size compared with stride 1.

2. **Compute the trainable parameters of a convolution layer having 64 filters of size 3×3 and an RGB input.**

Since the input image is RGB, the number of input channels is 3. Each filter therefore contains $3 \times 3 \times 3$ weights and one bias term.

$$\text{Parameters per filter} = 3 \times 3 \times 3 + 1 = 28$$

For 64 filters,

$$\text{Total Parameters} = 28 \times 64 = \boxed{1792}$$

Therefore, the convolution layer has 1792 trainable parameters.

3. **Compare ReLU and Sigmoid activation functions.**

ReLU and Sigmoid were compared by keeping the CNN architecture, optimizer, number of epochs, and batch size the same and changing the activation function used in the hidden layers.

The ReLU model achieved a test accuracy of 61.53% with a training time of 308.71 seconds. The Sigmoid model achieved a slightly higher test accuracy of 64.31%, but required 339.93 seconds for training.

In this particular experiment, Sigmoid gave a 2.78 percentage-point higher test accuracy but took more time to train. Although ReLU is generally used more often in CNN hidden layers because it is computationally simple and helps reduce the vanishing-gradient problem, the actual results of this experiment show that Sigmoid performed slightly better on the test set.

Activation	Test Accuracy	Training Time (s)
ReLU	61.53%	308.71
Sigmoid	64.31%	339.93

Table 1: Comparison of ReLU and Sigmoid activation functions.

4. **Replace Max Pooling with Average Pooling and compare the results.**

For this experiment, the MaxPooling layers were replaced with AveragePooling layers while keeping the other CNN settings unchanged.

The Max Pooling model achieved a test accuracy of 61.58% and took 392.29 seconds to train. The Average Pooling model achieved a slightly higher test accuracy of 62.18%, but its training time was higher at 486.61 seconds.

Therefore, Average Pooling improved the test accuracy by about 0.60 percentage points in this experiment, but it also required more training time. This shows that the choice of pooling operation can affect both model performance and computational time.

Pooling Method	Test Accuracy	Training Time (s)
Max Pooling	61.58%	392.29
Average Pooling	62.18%	486.61

Table 2: Comparison of Max Pooling and Average Pooling.

5. **Increase the number of convolution filters from 16 to 64 and analyze the effect on accuracy and computation time.**

The number of filters was increased from 16 to 64 while keeping the remaining CNN configuration the same. The 16-filter model achieved a test accuracy of 54.03% and required 273.11 seconds for training. It had 135,258 trainable parameters.

The 64-filter model achieved a higher test accuracy of 60.02%, but the training time increased to 706.04 seconds. The number of trainable parameters also increased to 564,426.

Thus, increasing the number of filters improved the test accuracy by about 5.99 percentage points, but it also increased the number of parameters and training time considerably. This shows the trade-off between increasing the feature-extraction capacity of the CNN and the additional computational cost.

Metric	16 Filters	64 Filters	Change
Test Accuracy	54.03%	60.02%	+5.99 pp
Training Time (s)	273.11	706.04	+432.93 s
Parameters	135,258	564,426	+429,168

Table 3: Effect of increasing convolution filters from 16 to 64.

12. Results

Metric	Value
Training Accuracy	94.57%
Testing Accuracy	60.63%
Precision	0.6103
Recall	0.6063
F1-score	0.6028
Number of Parameters	545,098

Table 4: Performance of the CNN Model

13. Discussion

1. Why is convolution preferred over fully connected layers for images?

Convolution is preferred because images contain spatial relationships between nearby pixels. A convolution layer looks at small regions of the image and learns features such as edges, textures, and shapes. It also uses the same filter at different locations, which helps the network detect the same feature wherever it appears in the image. A fully connected layer connects every input pixel to every neuron, resulting in a much larger number of parameters and losing the local spatial structure of the image.

2. How does stride affect the feature map size?

Stride determines how far the convolution filter moves over the input image at each step. With a smaller stride, the filter makes more overlapping operations and produces a larger feature map. Increasing the stride reduces the number of positions at which the filter is applied, so the feature map becomes smaller. For example, with a 3×3 filter and valid padding on a 32×32 image, the output size is 30×30 for stride 1 and 15×15 for stride 2.

3. What is the role of padding?

Padding consists of adding pixels, generally zeros, to the borders of the input image before performing the convolution operation. This makes it easier to manage the size of the output and enables pixels that lie on the boundaries of the image to take part in the convolution operation. Using the *same* padding, it is possible to maintain the size of the output similar to that of the input, when using stride 1.

4. Why is pooling used?

Pooling is done in order to bring down the dimensions in the spatial domain while preserving key information. It reduces computational complexity as well as the number of values required for processing by subsequent layers. The max-pooling method preserves the maximum value from the local neighborhood whereas average pooling takes the average value of the region. In this case study, a 2×2 pooling window with stride 2 resulted in a reduction of around half the spatial dimensions.

5. How do feature maps represent image characteristics?

Feature map shows how the chosen convolutional filter reacts to various parts of an image. The stronger the reaction, the more probable that the feature found out by this filter is presented there. The lower-level feature maps can show some simple features of the picture like edges, corners, and textures. With the help of additional convolutional layers, the neural network learns to unite these features and get more complicated shapes.

6. Why do CNNs require fewer parameters than MLPs?

Convolutional neural networks have less number of parameters because they implement localized connectivity and weight sharing. The convolution kernel is used multiple times on various regions of the image and so the same weight parameters are used again and again rather than using a new set of parameters for each individual connection between pixels and neurons. On the other hand, an MLP having fully connected layers will have all pixels of the input image connected to all neurons, which means a lot of parameters.

14. Conclusion

This experiment demonstrated the working principles of Convolutional Neural Networks for image classification using the CIFAR-10 dataset. The effects of different kernel sizes, stride values, and padding methods on the dimensions of convolutional feature maps were analyzed. Feature map visualization also demonstrated how different convolution filters respond to different local patterns and structures present in the input images.

In addition, the experiment showed the importance of pooling in making the spatial dimensions smaller without losing any crucial data in the process. Max Pooling and Average Pooling made feature maps with identical spatial dimensions whenever they were performed under the same parameters; however, their accuracy in classification was not identical. In the CNN, training involved convolution, ReLU, pooling, flattening, and fully connected layers, and testing was done through accuracy, precision, recall, F1-score, and confusion matrix.

The results of these additional experiments confirmed the impact of using different activation functions, different pooling approaches, as well as the number of convolutional filters on the performance of the neural network. In particular, the comparison of ReLU and Sigmoid revealed the difference in accuracy and training speed, and the comparison of Max Pooling and Average Pooling revealed the dependence of classification on the pooling operation itself. The increase of the number of

convolution filters helped to improve the accuracy of the test in the conducted experiment, but significantly increased the number of trainable parameters and processing time.

15. References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. TensorFlow Documentation.
5. CIFAR-10 Dataset Documentation.